

Michael — game2401 project feedback

Where your project is

Your project is House of Healing, and the specification behind it is genuinely strong: four systems each with one responsibility, SOLID reasoned against your own design rather than recited, and every pattern given a rationale and a trade-off. The catch is the gap between that document and the build. The processing layer follows the spec and runs, but the chemistry core, the part the whole game turns on, is a different design from the one you wrote, and its event chain is commented out and never fires. Since A1 the programming has not moved, and the time since has gone to the tech-art course. So the work now is not new systems, it is getting the game you already specified to actually run.

Feedback on the work so far

What is working. The processing pipeline is real and correct. `ProcessingSystem` picks the right strategy by ingredient category, calls `Execute`, and raises `OnIngredientProcessed`, and the ingredient's state actually flips. Your interface segregation is exactly as you argued it, with `IIngredient` and `IProcessable` kept separate so each system only sees what it needs. That is the Strategy pattern and clean SOLID applied properly, and it is the hard part done right.

The chemistry core does not run. This is the centre of the game, and it is the piece that is missing. The `Evaluate` method your spec describes throws `NotImplementedException`. What actually runs is a different design, a bench that auto-pairs ingredients on collision and spawns a result prefab, with no Success, Neutral, or Fail outcome and no event. You also have a second, player-facing workbench whose combine call is commented out. So there are two chemistry designs sitting side by side, and neither is finished.

The event chain is commented out. Your headline pattern, chemistry raising an event that the inventory and cleaning systems listen for, is not wired. There is no `OnCombinationResolved` event, and the subscriptions in both `InventorySystem` and `CleaningSystem` are commented out, so a success never reaches the inventory and a mess is never created. The `CombinationLookup` you wrote has its setup commented out too, so it always returns nothing.

Nothing has been stood up and tested. There is one general Playground scene and no test scene per system. This matters, because a gym for the chemistry system, run once with a couple of ingredients, would have shown you straight away that the chain never fires. The code reads clean, but clean code that has not been run is not the same as working code. You are clearly working with an LLM, which is good, and it raises the bar: the help should be closing the distance between your strong spec and a build that runs it, and a gym per system is how you check that it has.

The build and the spec describe two different games. Your spec rules out a player character for all three assignments, but the build has a full 3D player, camera, and pickup system. That is real, working code, but it means the document and the game no longer match. Decide which game this is, and bring the spec back in line with it.

What to build for A2 (due 22 July)

The aim is not more systems, it is the game you already specified, running and proven. Start by standing one system up in a test scene and running it, chemistry first: a scene with a few ingredients and your rules, played once so you can watch it resolve. Then settle on one chemistry design, finish it so **Evaluate** produces Success, Neutral, or Fail with the raw-ingredient block you specified, and delete the other. Then wire the event chain so a success reaches the inventory and a mess reaches cleaning, since the handlers are already written and just need connecting. Once the chain feeds it, the inventory wants a real display kept separate from the data behind it, and the crafting actions are a natural place to make actions undoable. A2 asks for the mechanics playable, a pattern applied to each with your reasoning written into the code, the specification updated as the build forces decisions, and a commit history that shows the iteration.

Your mechanics

Mechanic	Where it is now	Keep / Cut	Pattern to use
Processing (raw to processed)	Working. Strategy picks per category and the state flips.	Keep, working	Strategy
Chemistry core (outcome evaluation)	Attempted. Evaluate throws, and a different auto-trigger design runs instead.	Keep, finish first	—
Event chain (chemistry to inventory and cleaning)	Absent. No event, and both subscriptions commented out.	Keep, wire	Events / Observer
Building inventory of remedies	Stubbed. Basic storage works, the remedy inventory receives nothing.	Keep	MVC

Mechanic	Where it is now	Keep / Cut	Pattern to use
Cleaning	Stubbed. Clears a flag, but no mess is ever made.	Keep, wire	Events / Observer
Ingredient and rule data (ScriptableObjects)	Working. Authored and read by the chemistry path.	Keep	—
CombinationLookup	Stubbed. Setup commented out, and unused.	Decide: wire or cut	—
Gyms (a test scene per system)	One general Playground scene only.	Build	—
Undoable actions	Absent.	Keep, later	Command
Player, camera, interaction, pickup	Built and working, but out of your spec's scope.	Decide with the front-end	—
Outcome feedback for the player	Absent, and cannot show until the chain resolves.	Keep, once it runs	—
Win / lose	Absent, and not stated in your spec.	Decide, a design question	—
Pause	Absent.	Later	—
Title / main menu	Absent.	Later	—
Tutorial / onboarding	Absent.	Later	—
Settings	Absent.	Later	—
Audio	Absent.	Later	—
Save / load	Absent.	Later	—

Where to start

- **Stand up a chemistry gym first and run it.** A scene, a few ingredients, your rules, played once. This is the habit to build, and it shows you what does and does not fire.
- **Settle on one chemistry design.** Keep the auto-trigger bench or the staged workbench, finish it so `Evaluate` returns the outcome, and delete the other.

- **Wire the event chain.** Add the `OnCombinationResolved` event and uncomment the subscriptions so a success reaches the inventory and a mess reaches cleaning. The code is written, it needs connecting.
- **Give the inventory a real display once the chain feeds it,** keeping the view separate from the data behind it. Undoable actions can follow once the core loop runs.
- **Keep the spec honest to the build as you go, and apply a pattern to each mechanic with the reasoning in the code.** Both are A2 deliverables, marked as much as the working build.

Let's talk about

- **Which game this is.** The 3D player and world you built, or the text-driven management sim in your spec. Both can work, but pick one and make the spec match, because it changes what the rest of the code looks like.
- **The two chemistry designs.** You have a bench that auto-combines and a workbench you stage by hand. Decide which one is the game before finishing either.
- **Testing as you build.** A small gym per system, run as you go, is what keeps a strong architecture honest. It is the habit worth locking in now.